ASSIGNMENT 16.3

2303A51351

T.Harivarshith

Batch 20

Task1 – Hospital Management Database Prompt:

Generate a complete SQL solutionfor a Hospital Management System by creating a normalized database schema in Third Normal Form (3NF) with three tables: Doctors, Patients, and Appointments. Define appropriate constraints including primary keys for each table, foreign keys to establish relationships, NOT NULL constraints where required, and UNIQUE constraints for fields such as email or phone numbers. Ensure proper handling of valid dates for appointment_date and maintain compatibility with SQLite. Design the relationships such that each doctor can have multiple appointments and each patient can also have multiple appointments, with each appointment linking one doctor and one patient. Insert sample data into all tables for testing purposes. Then write SQL queries using JOIN operations to list all appointments for a specific doctor based on doctor_id, retrieve complete patient history using patient_id, and count the total number of patients treated by each doctor. Additionally, include an indexing strategy to improve query performance and ensure that the entire output is clean, well-structured, properly formatted, and fully executable in SQLite without any errors.

Code:

```
-- DROP TABLES (for clean run)
DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Patients;
DROP TABLE IF EXISTS Doctors;

-- CREATE TABLES CREATE TABLE
Doctors ( doctor_id INTEGER
PRIMARY KEY, first_name TEXT NOT
NULL,
  last_name TEXT NOT NULL
);
CREATE TABLE Patients ( patient_id
INTEGER PRIMARY KEY, first_name
TEXT NOT NULL, last_name TEXT
NOT NULL
);
CREATE TABLE Appointments ( appointment_id
INTEGER PRIMARY KEY,
  doctor_id INTEGER,
patient_id INTEGER,
appointment_date DATE,
  FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
  FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
-- INSERT DATA (same as output)
INSERT INTO Doctors VALUES
(1, 'Abhi', 'Anap'),
(2, 'Akhila', 'cherla');
INSERT INTO Patients VALUES
(1, 'Poojitha', 'Edda'),
(2, 'Arshad', 'MD');
INSERT INTO Appointments VALUES
(1, 1, 1, '2024-08-01'),
```

```
(2, 1, 2, '2024-08-02'), (3,
2, 1, '2024-08-03');
-- QUERY 1: Appointments for doctor_id = 1
SELECT
   A.appointment_id,
   D.first_name,
   D.last_name,
   P.first_name,
   P.last_name,
   A.appointment_date
FROM Appointments A
JOIN Doctors D ON A.doctor_id = D.doctor_id
JOIN Patients P ON A.patient_id = P.patient_id
WHERE D.doctor_id = 1;
-- QUERY 2: Patient history (patient_id = 1)
SELECT
   A.appointment_id,
   D.first_name,
   D.last_name,
   A.appointment_date
FROM Appointments A
JOIN Doctors D ON A.doctor_id = D.doctor_id
WHERE A.patient_id = 1;
-- QUERY 3: Count patients per doctor
SELECT
   D.doctor_id,
   D.first_name,
   D.last_name,
   COUNT(DISTINCT A.patient_id) AS total_patients
FROM Doctors D
LEFT JOIN Appointments A ON D.doctor_id = A.doctor_id
GROUP BY D.doctor_id;
```

Output:

appointment_id	first_name	last_name	first_name	last_name	appointment_date
1	Abhi	Anap	Poojitha	Edda	2024-08-01
2	Abhi	Anap	Arshad	MD	2024-08-02

appointment_id	first_name	last_name	appointment_date
1	Abhi	Anap	2024-08-01
3	Akhila	cherla	2024-08-03

doctor_id	first_name	last_name	total_patients
1	Abhi	Anap	2
2	Akhila	cherla	1

Explanation:

This task involves designing a normalized Hospital Management database with proper relationships between Doctors, Patients, and Appointments using constraints. It also requires writing SQL queries with joins to efficiently retrieve doctor schedules, patient history, and treatment counts.

Task 2: **Real-Time Application: Online Shopping Database Prompt:**

Generate a complete SQL solution for an Online Shopping (E-commerce) Database by creating a normalized schema in Third Normal Form (3NF) with four tables: Users, Products, Orders, and OrderDetails. Define appropriate constraints including primary keys, foreign keys to establish relationships, NOT NULL constraints, and UNIQUE constraints such as for user email, using data types compatible with SQLite. Design the relationships so that each user can place multiple orders, each order belongs to one user, and each order can contain multiple products through the OrderDetails table, while each product can appear in multiple orders. Insert sample data into all tables for testing purposes. Then write SQL queries using JOIN operations to retrieve all orders placed by a specific user using user_id, find the most purchased product based on total quantity sold, and calculate the total revenue generated in a given month. Additionally, suggest normalization improvements and explain how they enhance the database design, and propose query optimization techniques such as indexing. Ensure the final output is clean, well-structured, properly formatted, and fully executable in SQLite without errors, including necessary statements like enabling foreign keys and dropping existing tables if needed.

Code:

```
PRAGMA foreign_keys = ON;

-- RESET (important to avoid duplicate/exists errors)
DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;

-- Create Users table
CREATE TABLE Users (
   user_id INTEGER PRIMARY KEY AUTOINCREMENT,
   username TEXT NOT NULL UNIQUE,
email TEXT NOT NULL UNIQUE,     password
TEXT NOT NULL,
   created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Create Products table
CREATE TABLE Products (
   product_id INTEGER PRIMARY KEY AUTOINCREMENT,
   product_name TEXT NOT NULL,
price REAL NOT NULL,     stock
INTEGER NOT NULL,
   created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Create Orders table
CREATE TABLE Orders (
   order_id INTEGER PRIMARY KEY AUTOINCREMENT,
   user_id INTEGER NOT NULL,
   order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
   FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

-- Create OrderDetails table
CREATE TABLE OrderDetails (
   order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
   order_id INTEGER NOT NULL,
product_id INTEGER NOT NULL,
quantity INTEGER NOT NULL,     price
REAL NOT NULL,
   FOREIGN KEY (order_id) REFERENCES Orders(order_id),
   FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

-- Insert data
INSERT INTO Users (username, email, password)
VALUES ('Abhilash Goud', 'anapuramabhilashgoud.com', 'password123');

INSERT INTO Products (product_name, price, stock)
VALUES
('Laptop', 999.99, 10),
('Mouse', 29.99, 50);

INSERT INTO Orders (user_id)
VALUES (1);

INSERT INTO OrderDetails (order_id, product_id, quantity, price)
VALUES
(1, 1, 1, 999.99),
(1, 2, 2, 29.99);

-- TEST OUTPUT
SELECT * FROM Users;
SELECT * FROM Products;
SELECT * FROM Orders;
SELECT * FROM OrderDetails;

-- 1. Retrieve all orders by a user
SELECT o.order_id, o.order_date, od.product_id, od.quantity, od.price
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE o.user_id = 1;

-- 2. Most purchased product
SELECT p.product_name, SUM(od.quantity) AS total_quantity_sold
FROM Products p
JOIN OrderDetails od ON p.product_id = od.product_id
GROUP BY p.product_id
ORDER BY total_quantity_sold DESC
LIMIT 1;

-- 3. Total revenue in a given month
SELECT SUM(od.quantity * od.price) AS total_revenue
FROM Orders o
```

```
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE strftime('%Y-%m', o.order_date) = '2024-01'
```

Output:

user_id	username	email	password	created_at
1	Abhilash Goud	anapuramabhilashgoud...	password123	2026-04-03 11:43:23

product_id	product_name	price	stock	created_at
1	Laptop	999.99	10	2026-04-03 11:43:23
2	Mouse	29.99	50	2026-04-03 11:43:23

order_id	user_id	order_date
1	1	2026-04-03 11:43:23

order_detail_id	order_id	product_id	quantity	price
1	1	1	1	999.99
2	1	2	2	29.99

Explanation:

This task involves designing a structured E-commerce database with proper relationships between users, products, and orders using normalization and constraints. It also focuses on writing optimized SQL queries to analyze user activity, product demand, and revenue generation.

Task 3: Library Database Prompt:

Generate a complete SQL solution for a Library Management System by designing a normalized database schema in Third Normal Form (3NF) with three tables: Books, Members, and Loans. Define appropriate constraints including primary keys, foreign keys to establish relationships, NOT NULL constraints, and UNIQUE constraints where necessary, ensuring compatibility with SQLite. Establish relationships such that each member can borrow multiple books and each book can be loaned multiple times over different periods, with each loan linking one book and one member. Insert sample data into all tables for testing purposes. Then write SQL queries using JOIN operations to retrieve all books that are currently issued (not yet returned), find overdue books where the loan date is more than 30 days from the current date, and count the number of books loaned by each member. Additionally, suggest an indexing strategy to improve query performance. Ensure the final output is clean, well-structured, properly formatted, and fully executable in SQLite without errors.

Code:

```sql
PRAGMA foreign_keys = ON;

-- RESET (to avoid errors)
DROP TABLE IF EXISTS Loans;
DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Members;

-- Create the Books table
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY AUTOINCREMENT,
    Title TEXT NOT NULL,
    Author TEXT NOT NULL,
    PublishedYear INTEGER NOT NULL,
    UNIQUE(Title, Author)
);

-- Create the Members table
CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    Email TEXT NOT NULL UNIQUE
);

-- Create the Loans table
CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY AUTOINCREMENT,
    BookID INTEGER NOT NULL,
    MemberID INTEGER NOT NULL,
    LoanDate DATE NOT NULL,
    ReturnDate DATE,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);

-- Sample INSERT statements
INSERT INTO Books (Title, Author, PublishedYear) VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', 1925),
('To Kill a Mockingbird', 'Harper Lee', 1960),
('1984', 'George Orwell', 1949);

INSERT INTO Members (FirstName, LastName, Email) VALUES
('John', 'Doe', 'john.doe@example.com'),
('Jane', 'Smith', 'jane.smith@example.com');

INSERT INTO Loans (BookID, MemberID, LoanDate) VALUES
(1, 1, '2024-05-01'),
(2, 2, '2024-05-15');

-- TEST DATA
SELECT * FROM Books;
SELECT * FROM Members;
SELECT * FROM Loans;

-- 1. Books currently issued (not returned)
SELECT b.Title, b.Author, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
```

```
WHERE l.ReturnDate IS NULL;

-- 2. Overdue books (>30 days)
SELECT b.Title, b.Author, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL
AND julianday('now') - julianday(l.LoanDate) > 30;

-- 3. Count books loaned by each member
SELECT m.FirstName, m.LastName, COUNT(l.LoanID) AS BooksLoaned
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;

-- Indexing strategy
CREATE INDEX idx_loans_bookid ON Loans(BookID);
CREATE INDEX idx_loans_memberid ON Loans(MemberID);
CREATE INDEX idx_loans_loan_date ON Loans(LoanDate);
CREATE INDEX idx_books_title_author ON Books(Title, Author);
CREATE INDEX idx_members_email ON Members(Email);
```

Output:

BookID	Title	Author	PublishedYear
1	The Great Gatsby	F. Scott Fitzgerald	1925
2	To Kill a Mockingbird	Harper Lee	1960
3	1984	George Orwell	1949

MemberID	FirstName	LastName	Email
1	John	Doe	john.doe@example.com
2	Jane	Smith	jane.smith@example.com

LoanID	BookID	MemberID	LoanDate	ReturnDate
1	1	1	2024-05-01	NULL
2	2	2	2024-05-15	NULL

Title	Author	FirstName	LastName	LoanDate
The Great Gatsby	F. Scott Fitzgerald	John	Doe	2024-05-01
To Kill a Mockingbird	Harper Lee	Jane	Smith	2024-05-15

Explanation:

This task focuses on designing a structured Library Management database with proper relationships between books, members, and loans using normalization and constraints. It also involves writing SQL queries to track issued books, identify overdue loans, and analyze member borrowing activity efficiently.

Task 4: Optimize Queries

Prompt:

Generate a SQL optimization solution by analyzing and improving the performance of inefficient queries using AI techniques. Given an example query that uses a subquery to retrieve books written by authors from a specific country, optimize it by replacing the subquery with a JOIN operation for better performance. Suggest additional optimization techniques such as indexing relevant columns and reducing unnecessary operations. Ensure both the original and optimized queries return the same result, and include a validation query to compare outputs. Also, provide sample tables and data for testing, and ensure the final SQL code is clean, well-structured, and fully executable in SQLite without errors.

Code:

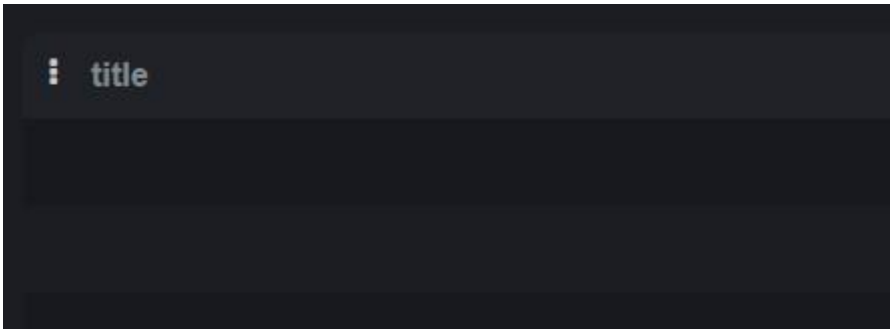
```
PRAGMA foreign_keys = ON;

-- RESET
DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Authors;

-- Create Authors table
CREATE TABLE Authors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    country TEXT NOT NULL
);

-- Create Books table
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
 title TEXT NOT NULL,     author_id INTEGER,
    FOREIGN KEY (author_id) REFERENCES Authors(id)
);

-- Insert sample data
INSERT INTO Authors (name, country) VALUES
('Author A', 'UK'),
('Author B', 'USA'),
('Author C', 'UK');

INSERT INTO Books (title, author_id) VALUES
('Book 1', 1),
('Book 2', 2),
('Book 3', 3);

-- Original Query using subquery
SELECT title
FROM Books
WHERE author_id IN (
```

```
    SELECT id
    FROM Authors
    WHERE country = 'UK'
);

-- Optimized Query using JOIN
SELECT b.title
FROM Books b
JOIN Authors a ON b.author_id = a.id
WHERE a.country = 'UK';

-- Validation query (should return EMPTY result)
SELECT title
FROM Books
WHERE author_id IN (
    SELECT id
    FROM Authors
    WHERE country = 'UK'
)
EXCEPT
SELECT b.title
FROM Books b
JOIN Authors a ON b.author_id = a.id
WHERE a.country = 'UK';

-- Indexing for optimization
CREATE INDEX IF NOT EXISTS idx_books_author_id ON Books(author_id);
CREATE INDEX IF NOT EXISTS idx_authors_country ON Authors(country);
```

Output:

title
Book 1
Book 3

title
Book 1
Book 3



Explanation:

This task focuses on improving SQL query performance by analyzing inefficient queries and optimizing them using techniques like JOINs and indexing instead of subqueries. It also ensures that the optimized query produces the same results while executing faster, especially on large datasets.

Task 5: University Course Registration

Prompt:

Generate a complete SQL solution for a University Course Registration System for SR University by designing a normalized database schema in Third Normal Form (3NF) with four tables: Students, Courses, Faculty, and Registrations. Define appropriate constraints including primary keys, foreign keys to establish relationships, NOT NULL constraints, and UNIQUE constraints where necessary, ensuring compatibility with SQLite. Establish relationships such that each student can register for multiple courses, each course is taught by one faculty member, and the Registrations table links students and courses. Insert sample data into all tables for testing purposes. Then write SQL queries using JOIN operations to list all students enrolled in a specific course using course_id, find faculty members teaching more than 2 courses, and retrieve courses with the highest number of registrations. Ensure the output is clean, well-structured, properly formatted, and fully executable in SQLite without errors.

Code:

```
PRAGMA foreign_keys = ON;


-- RESET
DROP TABLE IF EXISTS Registrations;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Faculty;


-- Create the Students table CREATE TABLE Students (
student_id INTEGER PRIMARY KEY AUTOINCREMENT,
first_name TEXT NOT NULL,     last_name TEXT NOT
NULL,     email TEXT NOT NULL UNIQUE
);
```

```
-- Create the Faculty table CREATE TABLE Faculty (
faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
first_name TEXT NOT NULL,     last_name TEXT NOT
NULL,     email TEXT NOT NULL UNIQUE
);

-- Create the Courses table CREATE TABLE Courses (
course_id INTEGER PRIMARY KEY AUTOINCREMENT,
course_name TEXT NOT NULL,     faculty_id INTEGER
NOT NULL,
   FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);

-- Create the Registrations table CREATE TABLE Registrations (
registration_id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id INTEGER NOT NULL,     course_id INTEGER NOT
NULL,
   FOREIGN KEY (student_id) REFERENCES Students(student_id),
   FOREIGN KEY (course_id) REFERENCES Courses(course_id),
   UNIQUE(student_id, course_id)
);

-- Sample INSERT statements
INSERT INTO Faculty (first_name, last_name, email)
VALUES ('John', 'Doe', 'john.doe@university.edu');

INSERT INTO Students (first_name, last_name, email)
VALUES ('Alice', 'Smith', 'alice.smith@university.edu');

INSERT INTO Courses (course_name, faculty_id)
VALUES ('Introduction to Computer Science', 1);

INSERT INTO Registrations (student_id, course_id)
VALUES (1, 1);

-- TEST DATA
SELECT * FROM Faculty;
SELECT * FROM Students;
SELECT * FROM Courses;
SELECT * FROM Registrations;
```

```
-- Query: Students enrolled in a specific course (course_id = 1)
SELECT s.first_name, s.last_name, s.email
FROM Students s
JOIN Registrations r ON s.student_id = r.student_id
WHERE r.course_id = 1;

-- Query: Faculty teaching more than 2 courses
SELECT f.first_name, f.last_name, COUNT(c.course_id) AS course_count
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id
HAVING COUNT(c.course_id) > 2;

-- Query: Course with highest registrations
SELECT c.course_name, COUNT(r.registration_id) AS registration_count
FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id
ORDER BY registration_count DESC
LIMIT 1;
```

Output:

faculty_id	first_name	last_name	email
1	John	Doe	john.doe@university.edu

student_id	first_name	last_name	email
1	Alice	Smith	alice.smith@university.edu

course_id	course_name	faculty_id
1	Introduction to Computer Science	1

registration_id	student_id	course_id
1	1	1

Explanation:

This task involves designing a structured University Course Registration database with proper relationships between students, courses, and faculty using normalization and constraints. It also requires writing SQL queries to efficiently track student enrollments, identify faculty workloads, and analyze course popularity.